

第3章 流程控制

建议学时:3-4 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 Rust 的全部流程控制原语:if/else、match、if let、while let、loop、while、for——与 C 的 if/switch/for/while/do-while/goto 逐一对照
- 理解 match 相对 C 的 switch 的三个飞跃:穷尽性检查、模式绑定、守卫条件;C 的 fallthrough 与"只能整型"两大致命缺陷在 Rust 中如何被消灭
- 掌握 if let/while let 语法糖,以及 let-else 现代写法
- 掌握三种循环的定位:loop(无限)、while(条件)、for(遍历);会用 loop+if break 模拟 C 的 do-while
- 掌握 break 带值、continue、带标签 break 'outer——Rust 对 C 的 goto 的优雅替代
- 掌握 for 与迭代器:0..n 范围遍历、iter()/iter_mut()/into_iter() 三形态、循环变量每轮新绑定(没有 Go 1.22 前的经典坑)
- 理解"遍历中修改容器"的迭代器失效问题在 Rust 中如何被借用检查器在编译期拦截

💡 从 C 到 Rust:本章主题如何迁移

C 的 switch 有两个与生俱来的缺陷:一是 fallthrough——忘记写 break 就掉进下一个分支,这是 C 生产代码里最经典的 bug 之一;二是它只能对整数做等值比较。Rust 的 match 把这两个缺陷一起消灭:分支之间自动隔离(没有 fallthrough),而且能对任意类型做"模式匹配"——解构结构体、匹配区间、绑定变量、加守卫条件。你在 C 里用一长串 else-if 和 switch 拼出来的逻辑,在 Rust 里是一段整洁的 match。

Rust 对"非穷尽"采取零容忍:match 必须覆盖所有可能,否则编译失败。这看似苛刻,实则是最有价值的安全网——C 里"忘了写 default"只是默默漏掉一种情况;Rust 里编译器会指着那一种情况问你:它该怎么办?你被迫把每种输入都安排明白,这正是生产代码需要的严谨。

C 的 goto 被视为禁忌:滥用会写出意大利面条式代码,但某些场景(深层嵌套中跳出)确实需要它。Rust 的态度是"用更好的原语替代":带标签的 break 'outer 让"跳出两层循环"有了安全优雅的写法;提前返回 让错误处理不再需要跳转。goto 的能力被保留,混乱被移除。

最后是循环哲学:C 的 for 是"下标驱动的循环"(i 从 0 到 n),Rust 的 for 是"迭代器驱动的循环"(依次取每个元素)。前者把注意力放在"怎么数",后者把注意力放在"要什么"——配合 0..n 范围与迭代器方法链,大多数循环在 Rust 里更短、更不易错,而且编译器保证你不会写错边界。

📦 前置知识自查

- C 的 if/else、switch/case/break/default、for/while/do-while 的完整语义
- C 的 fallthrough 陷阱、悬挂 else、for(;;) 无限循环
- C 的 goto 与"提前返回"两种跳出风格
- C 的数组遍历 for (i = 0; i < n; i++) 与指针遍历
- 第1章内容:Option、元组、Vec、枚举;第2章内容:if 是表达式、.. 范围

🚀 本章学习路线

1. **第一阶段(约1小时):**§3.2–§3.3,重点体验 match 的穷尽性与模式绑定,把 C 的 switch 习惯改过来
2. **第二阶段(约1小时):**§3.4–§3.5,练习三种循环与带标签 break、break 带值
3. **第三阶段(约1.5小时):**§3.6 迭代器与 for 的三种形态,再完成章末实验
4. **验收标准:**能用 match 重写任意 if-else 链;能写出"跳出两层循环"与"至少执行一次"的代码;对每个 C 流程控制语句能说出 Rust 对应物

本章知识导图

- §3.1 总览:流程控制差异清单
- §3.2 if 与 if 表达式:条件必须 bool、无悬挂 else
- §3.3 match:C 的 switch 进化版(穷尽/绑定/守卫/区间/多模式)
- §3.4 if let 与 while let:Option 的语法糖
- §3.5 三种循环:loop / while / for、do-while 模拟、break 带值、标签
- §3.6 for 与迭代器:0..n、iter/iter_mut/into_iter、循环变量新绑定
- §3.7 goto 的替代:提前返回与标签 break;迭代器失效问题
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§3.1 总览:与 C 的差异清单

C 的写法	Rust 的写法	差异要点
if (cond) {...} else {...}	if cond {...} else {...}	无括号;条件必须是 bool
switch (n) { case 1: ... }	match n { 1 => ..., _ => ... }	无 fallthrough;必须穷尽;可匹配任意类型
for (i = 0; i < n; i++)	for i in 0..n	范围迭代,编译器保证边界
while (cond)	while cond	相同
do {...} while (cond)	loop {...; if !cond { break } }	Rust 无 do-while,用 loop+break 模拟
for(;;) / while(1)	loop	无限循环专用关键字
break / continue	break / continue	相同;另支持带标签与 break 带值
goto label	'label: loop {...} + break 'label	仅用于跳出嵌套循环,比 goto 安全

§3.2 if 与 if 表达式

3.2.1 条件必须 bool;没有悬挂 else

第2章已强调: `if 1`、`if ptr` 都是编译错误。这里补一个 C 的经典坑的消失:C 的悬挂 else(else 就近配对被误读),在 Rust 里因为大括号必须成对出现而**根本不可能发生**——每个 if 的块边界清清楚楚:

```
fn main() {
    let hour = 9;
    // C 里缩进暗示、编译器就近配对的经典悲剧,在 Rust 中无法构造
    if hour < 12 {
        println!("上午");
    } else if hour < 18 {
        println!("下午");
    } else {
        println!("晚上");
    }
}
```

3.2.2 if 表达式的完整形态

第2章已见 if 表达式的"取两个值"形态;这里补两个生产细节:① if-else 链求值时,从上到下第一个条件成立的分支生效;② 只做副作用时,分支返回 ()——Rust 允许省略 else:

```
fn main() {
    let price = 120;
    // if 表达式:所有分支类型一致(这里是 &str)
    let label = if price >= 100 {
        "贵"
    } else {
        "便宜"
    };
    println!("{label}");

    // 只做副作用:允许没有 else(此时类型是 ())
    let mut cart = Vec::new();
    if price <= 200 {
        cart.push("奶茶");
    }
    println!("{:?}", cart);

    // 经典误用:C 的三目被嵌套使用—Rust 建议拆开
    let grade = if price > 80 { "高" } else { "低" };
    println!("{grade}");
}
```

⚠ 易错点 1:if 表达式分支类型不一致,或忘了"无分号才是值"

编译错误 "if and else have incompatible types" 与第2章讲过的 "expected i32, found ()" 是本章最高频的两个报错。检查顺序:① 每个分支的最后一个表达式有没有多写分号;② 分支是否真的返回同类型;③ 是不是忘了 else(缺 else 时类型是 (),而外面又把它当值用。三分钟内定位,永不慌张。

§3.3 match:C 的 switch 进化版

3.3.1 穷尽性:编译器逼你覆盖所有情况

match 的每个分支无需 break(自动隔离),但整个 match 必须**穷尽**所有可能——漏掉任何情况都是编译错误。对整数,要么列出所有可能性,要么用通配 `_` 收尾。与 C 对照:C 忘写 default 是"悄悄漏掉",Rust 忘写 `_` 是"编译失败,必须处

理":

```
fn main() {
    let n = 3;
    match n {
        0 => println!("零"),
        1 => println!("一"),
        // 注释掉下面一行试试:编译器报"非穷尽"
        _ => println!("其他数"),
    }

    // match 是表达式:整个 match 的值赋给变量
    let word = match n {
        0 => "zero",
        1 => "one",
        _ => "many",
    };
    println!("{word}");

    // 多模式组合:一个分支匹配多个值(C 里要连写几个 case)
    let c = 'x';
    match c {
        'a' | 'e' | 'i' | 'o' | 'u' => println!("元音"),
        'a'..'z' => println!("小写字母"), // 区间模式
        _ => println!("其他字符"),
    }
}
```

3.3.2 模式绑定与守卫

这是 match 相对 switch 的质变:模式可以"拆开"被匹配的值,并把其中一部分绑定为变量——C 的 switch 永远做不到,你只能 case 值然后手动取字段。守卫 `if` 给模式加上附加条件:

```
// 带数据的枚举(第1章见过):模式绑定在此发力
enum Shape {
    Circle(f64),
    Rect { w: f64, h: f64 },
}

fn area(shape: &Shape) -> f64 {
    match shape {
        // 绑定:把圆括号里的 f64 绑给 r
        Shape::Circle(r) => 3.14159 * r * r,
        // 结构体模式:字段名直接绑定
        Shape::Rect { w, h } => w * h,
    }
}

fn main() {
    let shapes = vec![Shape::Circle(2.0), Shape::Rect { w: 3.0, h: 4.0 }];
    for s in &shapes {
        println!("面积:{}", area(s));
    }

    // 守卫:模式之外再加条件
    let score = 85;
    match score {
        s if s >= 90 => println!("优秀"),
        s if s >= 80 => println!("良好"),
        s if s >= 60 => println!("及格"),
        _ => println!("不及格"),
    }
}
```

3.3.3 与 C 的 switch 对照表

对比项	C 的 switch	Rust 的 match
fallthrough	不写 break 就穿透(经典 bug 源)	分支自动隔离,无穿透
可匹配类型	只能整型(及 char 提升)	任意类型:枚举/结构体/元组/区间/引用
穷尽性	无检查,漏 case 或 default 静默	必须穷尽,否则编译错误
绑定变量	case 只做等值比较	模式绑定 + 守卫 if + 多模式
表达式性	语句	表达式,可直接产生值
default	default 分支	_ 通配(可绑定:_x 表示"要值但忽略")

⚠ 易错点 2:把 C 的 fallthrough 习惯带进 match,或误用 _ 遮蔽

① 以为"执行完这个分支继续下一个",不需要——Rust 分支之间没有穿透,写 `=> println!("a"); println!("b")` 是同一个分支的两条语句。② `_` 是"忽略所有未列出的值",`_x` 是"绑定但故意不用";在模式里把 `_` 当变量用会编译错误(`_` 不能读)。③ 忘了模式绑定是"借用还是移动"——match 一个 `&Shape` 时绑定的是引用,`r * r` 直接可用;match 拥有所有权时,`String` 等字段会被 move 走(第5章),之后就不能再用原值了。

§3.4 if let 与 while let

当 `match` 只有一个分支有意义时(最常见:检查 `Option` 有没有值),完整 `match` 显得冗长:`match opt { Some(v) => ..., None => () }`。语法糖 `if let` 专治这种"只想处理一种情况"的场景;匹配失败时可选 `else` 分支:

```
fn main() {
    let opt: Option<i32> = Some(7);
    // 完整 match
    match opt {
        Some(v) => println!("完整写法:{v}"),
        None => (),
    }
    // if let:同一逻辑的精简写法
    if let Some(v) = opt {
        println!("糖写法:{v}");
    }
    // 带 else:处理失败情况
    let none: Option<i32> = None;
    if let Some(v) = none {
        println!("{v}");
    } else {
        println!("没有值");
    }
}
```

while let:循环直到失败

```
fn main() {
    // 经典场景:从容器里不断取元素,直到取空
    let mut stack = vec![10, 20, 30];
    // C 的等价物:while (top != -1) { x = stack[top--]; }
    while let Some(v) = stack.pop() {
        println!("弹出:{v}");
    }
    println!("栈空了:{v}", stack.is_empty()); // true

    // 另一个场景:迭代器逐个取(第4章会细讲闭包与迭代器)
    let mut iter = [1, 2, 3].into_iter();
    while let Some(x) = iter.next() {
        print!("{x} ");
    }
    println!();
}
```

let-else:失败就提前退出(1.65+ 的现代写法)

```
fn parse_port(text: &str) -> u16 {
    // let-else:解构失败时执行 else 块并退出函数
    let Some(port) = text.trim().parse::<u16>().ok() else {
        println!("端口号无效,使用默认 8080");
        return 8080;
    };
    // 到这里 port 一定可用
    port
}

fn main() {
    println!("{}", parse_port("9999")); // 9999
    println!("{}", parse_port("abc"));  // 使用默认值 8080
}
```

工程建议:if let / while let / let-else 的选择

- 只有一种情况要处理:if let;两种都要处理:match(可读性高于 if-let-else)
- 解构失败意味着"函数无法继续":let-else,一目了然
- 循环消费一个 Option 序列:while let
- 生产代码里 if let Some(x) = map.get(key) 是最高频的写法之一,务必形成肌肉记忆

\$3.5 三种循环:loop / while / for

3.5.1 loop:无限循环专用

Rust 为"无限循环"专门造了 `loop` 关键字(C 的 `for(;;)` / `while(1)`),语义清晰;更重要的是 `break` 可以带值:loop 整体是一个表达式,退出时把值"算"出来——这是 C 无法表达的:

```
fn main() {
    // C 的 for(;;) / while(1) 在 Rust 里:
    loop {
        println!("无限循环(用 break 退出)");
        break;
    }

    // break 带值:loop 求出一个结果(C 需要临时变量 + 多次判断)
    let mut n = 1;
    let answer = loop {
        n += 1;
        if n * n > 1000 {
            break n; // 把 n 作为 loop 的值带出去
        }
    };
    println!("第一个平方大于 1000 的数是 {answer}({answer}^2 = {})", answer * answer);
}
```

3.5.2 while 与 do-while 模拟

```
fn main() {  
    // while:与 C 完全相同,条件为假可能一次都不执行  
    let mut i = 0;  
    while i < 3 {  
        print!("{i} ");  
        i += 1;  
    }  
    println();  
  
    // C 的 do-while 在 Rust 里没有,用 loop + if break 模拟:  
    // 先执行循环体,再检查条件  
    let mut tries = 0;  
    loop {  
        tries += 1;  
        println!("第 {tries} 次尝试");  
        if tries >= 3 {  
            break;           // 等价于 C 的 while(条件) 判断失败退出  
        }  
    }  
}
```

场景	C	Rust
无限循环	while (1) / for (;;)	loop
条件循环	while (cond)	while cond
至少执行一次	do { } while (cond)	loop { ...; if !cond { break; } }
固定次数	for (i = 0; i < n; i++)	for i in 0..n
遍历容器	for (p = head; p; p = p->next)	for x in iterable
跳出多层	goto 或标志位	'outer: loop + break 'outer

3.5.3 break/continue 与标签

```
fn main() {
    // continue:跳过本轮剩余部分,与 C 相同
    for i in 0..6 {
        if i % 2 == 0 {
            continue;           // 偶数跳过
        }
        print!("{i} ");         // 只打印奇数
    }
    println!();

    // 带标签 break:跳出外层循环(goto 的安全替代)
    let mut count = 0;
    'outer: for i in 1..=5 {
        for j in 1..=5 {
            count += 1;
            if i * j > 10 {
                println!("在 i={i}, j={j} 处跳出,共执行 {count} 次");
                break 'outer;    // 直接跳出外层 for
            }
        }
    }

    // 带标签 continue:跳到外层循环的下一轮迭代
    'filter: for i in 0..5 {
        for j in 0..5 {
            if i + j >= 4 {
                continue 'filter; // 跳过 i 的剩余 j,进入下一个 i
            }
            print!("{i},{j} ");
        }
    }
    println!();
}
```

⚠ 易错点 3:标签语法与 break 的误用

三个高频错误:① 标签写法是 `'outer:` (单引号在冒号左边),写在循环语句之前;写反或漏引号都编译错误。② `break` 不带标签时只能退出最内层循环——想退出两层,必须给外层加标签。③ `loop` 中 `break` 带值后,`loop` 的值类型必须一致; `let x = loop { break 1; break 2; }` 没问题,但 `break;` (无值)与 `break 1;` 混用会类型错误。

§3.6 for 与迭代器

3.6.1 范围遍历:C 的 for(i) 的现代替换

```
fn main() {  
    // C: for (int i = 0; i < n; i++)  
    // Rust:0..n 半开区间,编译器保证边界正确  
    for i in 0..5 {  
        print!("{i} ");  
    }  
    println();  
  
    // C: for (int i = 0; i <= n; i++)  
    for i in 0..=5 {  
        print!("{i} ");  
    }  
    println();  
  
    // 倒序:C 的 for (i = n-1; i >= 0; i--)——注意这里 C 的经典下溢坑  
    for i in (0..5).rev() {  
        print!("{i} ");  
    }  
    println();  
}
```

C 的 `for (i = n-1; i >= 0; i--)` 有一个著名的坑:`i` 是 `int` 时条件永远成立(下溢),必须改用 `size_t` 再小心处理;
`(0..5).rev()` 一行搞定且绝无此问题。

3.6.2 迭代器的三种形态:iter / iter_mut / into_iter

对容器 `v` 做 `for` 遍历有三种意图:只读(`&v / v.iter()`)、改写(`&mut v / v.iter_mut()`)、搬走(`v.into_iter()`)——对应第5章的借用三态,这里先看用法:

```
fn main() {
    let mut v = vec![1, 2, 3];

    // 形态一:只读遍历(推荐:for x in &v)
    for x in &v {
        print!("{x} ");
    }
    println();

    // 形态二:改写每个元素
    for x in &mut v {
        *x *= 10;           // 注意:此处 x 是 &mut i32,要解引用
    }
    println!("{v:?}");      // [10, 20, 30]

    // 形态三:把元素全部搬走(循环后 v 不可再用,第5章解释)
    let total: i32 = v.into_iter().sum();
    println!("总和:{total}");
    // println!("{v:?}");    // 编译错误:v 已被搬空
}
```

3.6.3 循环变量每轮都是新绑定

Go 1.22 之前的经典 bug:循环变量是同一个变量,闭包捕获的是它的地址,循环结束后全部指向最后一个值。Rust 的 `for i in 0..n` 中 **每一轮迭代 `i` 都是独立的新绑定**,捕获安全,此坑不存在:

```
fn main() {
    // 每轮 i 都是新绑定:收集到闭包里也各自独立(第4章讲闭包)
    let mut captured = Vec::new();
    for i in 0..5 {
        captured.push(move || i * i);    // 每个闭包拥有自己的 i
    }
    for f in &captured {
        print!("{}", f());
    }
    println();                          // 0 1 4 9 16
}
```

§3.7 goto 的替代与迭代器失效

3.7.1 Rust 没有 goto:替代方案清单

- **跳出嵌套循环**:带标签 `break/continue('outer: loop + break 'outer)`,覆盖 `goto` 90% 的合法用途
- **提前退出函数**:`return`——Rust 里"守卫 + 提前返回"是主流风格,见下方 `is_prime` 示例
- **错误处理跳转**:`?` 运算符(第2章)与 `match`(本章),替代 C 的 `goto err_cleanup` 模式
- **状态机逻辑**:`loop + match`(状态转移表),替代 `goto` 拼状态跳转

```
// 提前返回风格:C 的 goto 清理模式被"每层提前 return"替代
fn classify(n: i32) -> &'static str {
    if n < 0 {
        return "负数";        // 提前返回,无需 goto 跳转
    }
    if n == 0 {
        return "零";
    }
    if n % 2 == 0 {
        return "正偶数";
    }
    "正奇数"
}

fn main() {
    for n in [-3, 0, 4, 7] {
        println!("{n}: {}", classify(n));
    }
}
```

3.7.2 迭代器失效问题:C++ 的经典噩梦在 Rust 中不存在

C++ 里"遍历 vector 的同时 push/erase"会让迭代器失效,造成悬垂访问;C 里则是在遍历链表时删除节点,轻则跳过元素重则崩溃。Rust 的借用规则(第5章)在 **编译期** 直接禁止"一边借用一边修改":

```
fn main() {
    let mut v = vec![1, 2, 3];

    // 下面的代码编译不过:for x in &v 是共享借用,
    // v.push 需要可变借用,两者同时存在即冲突
    // for x in &v {
    //     v.push(*x);        // 编译错误:cannot borrow `v` as mutable
    // }

    // 正确做法:先算好增量,再一次性修改
    let doubled: Vec<i32> = v.iter().map(|x| x * 2).collect();
    v.extend(doubled);
    println!("{v:?}");      // [1, 2, 3, 2, 4, 6]
}
```

★ 重点:流程控制的 Rust 心法

- 能用表达式就不用语句;if/match/loop 优先"算出值",而不是"改变量"
- match 优先于 if-else 链:分支多、有数据要拆开、需要穷尽检查时,一律 match
- 循环优先 for + 迭代器;需要条件退出用 while;无限循环用 loop;do-while 用 loop+break
- 编译器是你最好的伙伴:穷尽性、借用冲突、边界错误——全都编译期发现

本章速查表

主题	结论 / 写法
if	条件必须 bool;无括号;if-else 是表达式,分支类型必须一致
match 分支	<code>n => expr</code> ;多模式 <code>a b</code> ;区间 <code>1..=9</code> ;守卫 <code>s if cond</code> ;通配 <code>_</code>
match 穷尽	必须覆盖所有可能,否则编译错误; <code>_</code> 处理剩余
模式绑定	<code>Some(v)</code> 、 <code>Circle(r)</code> 、 <code>Rect { w, h }</code> 把数据拆出来绑给变量
if let	<code>if let Some(v) = opt { ... } else { ... }</code> ;只关心一种情况的糖
while let	<code>while let Some(v) = stack.pop()</code> ;循环取到失败为止
let-else	<code>let Some(v) = f() else { return 默认值; }</code> ;失败提前退出
loop	无限循环;break 可带值,loop 整体是表达式
while	条件循环,可能一次都不执行
do-while	Rust 没有; <code>loop { 体; if !cond { break; } }</code> 模拟
for	<code>for i in 0..n</code> ; <code>for x in &v</code> (只读)/ <code>&mut v</code> (改写)/ <code>v</code> (搬走)
break/continue	break 退出当前循环;continue 跳下一轮;支持 'label: 标签
break 带值	<code>let ans = loop { ...; break 42; }</code>
goto	无;替代:标签 break、提前返回、? 运算符
迭代器失效	借用检查器编译期禁止"遍历中修改容器"
循环变量	每轮新绑定,闭包捕获无 Go 1.22 前的经典坑

最小必会清单

- 能把任意 if-else 链改写成 match,并说出 match 的三个相对 switch 的优势
- 能解释"match 必须穷尽"在生产中的价值,并写出 `_` 通配
- 能写出区间模式、多模式 `|`、带守卫 if 的 match 各一个
- 能区分 if let 与 match 的适用场景,并写 while let 消费一个 Option 序列
- 能写出三种循环各自的最佳场景,以及 do-while 的模拟写法
- 能写 break 带值取出 loop 的结果
- 能给两层循环加标签,并写出 `break 'outer` 与 `continue 'filter`
- 能写出 for 的三种形态(`iter`/`iter_mut`/`into_iter`)并说明区别
- 能解释为什么"遍历中修改容器"在 Rust 中编译不过
- 能说出 Rust 用哪些机制替代 C 的 goto,并各给一个例子

自测题

Q 1. 下面的 match 为什么编译错误? `let n = 5; match n { 1 => "一", 2 => "二" }` 如何修复?

✅ 参考答案

非穷尽: `n` 是 `i32`, 可以取无数个数, 只列出 1 和 2 无法覆盖, 编译器报 "non-exhaustive patterns"。修复: 加通配分支 `_ => "其他"` (或 `_x` 如果要用到它)。这正是 "编译器逼你处理所有情况" 的体现——C 里忘了 `default` 只是静默。

Q 2. C 的 `switch` 的 `fallthrough` 与 Rust 的 `match` 有什么关系? 在 `match` 里如何实现 "多个值共用一段代码"?

✅ 参考答案

`match` 分支之间自动隔离, 不存在 `fallthrough`——C 忘写 `break` 的 bug 在 Rust 不可能发生。多个值共用代码用多模式: `1 | 2 | 3 => ...`, 或区间 `1..=3 => ...`。注意: 分支内不能 "往下掉", 但模式本身可以罗列多个情况。

Q 3. 写一个 `loop` 表达式: 从 10 开始递减, 找到第一个能被 7 整除的数, 并把它作为 `loop` 的值。

✅ 参考答案

`let mut n = 10; let ans = loop { if n % 7 == 0 { break n; } n -= 1; };` 结果 7。要点: `break` 带值; `n` 是 `mut`; 若 `n` 为负会一直减——生产里应加边界条件 (这也说明 `break` 带值让 "循环求值" 成为一等公民)。

Q 4. C 的 `do-while` 循环如何用 Rust 改写? 给出 "至少执行一次" 的完整示例。

✅ 参考答案

`loop { 循环体; if !条件 { break; } }`。例如: `let mut t = 0; loop { t += 1; if t >= 3 { break; } }`。与 `while` 的区别: `while` 可能一次都不执行, 此写法保证至少执行一次循环体。

Q 5. 为什么 Rust 的 `for` 循环不存在 Go 1.22 之前的 "循环变量捕获" bug?

✅ 参考答案

Go 1.22 前, `for` 循环变量是循环外声明的一个变量, 每轮复用, 闭包捕获的是同一个地址, 循环后全部指向末值。Rust 的 `for i in 0..n` 每轮迭代创建全新的绑定, 闭包各自捕获独立的 `i`。语言设计层面消灭, 而非靠程序员小心。

Q 6. 如何在 Rust 里 "遍历数组的同时修改它"? 直接 `for x in &v` 里 `push` 会怎样?

✅ 参考答案

直接 `push` 编译错误: `for x in &v` 是共享借用, `push` 需要可变借用, 借用冲突 (第 5 章细讲)。正确做法: 先计算增量 (`let extra: Vec<_> = v.iter().map(...).collect();`) 再 `v.extend(extra)`。C++ 的迭代器失效在 Rust 中由编译器拦截。

章末实验题:文本菜单 + 素数筛选 + 数组统计

实验 3:菜单驱动的控制台工具集

实验目标:实现一个菜单驱动的控制台程序,包含素数筛选、数组统计、二维查找三个工具,强制使用 match 菜单、三种循环、break/continue 与带标签跳转。

实验要求:

- 任务 1:主循环用 loop + match 实现菜单分发,无效输入用 continue 回到菜单,退出用 break(覆盖:loop 无限循环、match 穷尽与 _ 通配、continue/break)
- 任务 2:is_prime 用 while 循环 + 提前返回实现(覆盖:while、goto 替代——提前返回)
- 任务 3:sieve 用 for i in 2..=n 范围遍历收集素数(覆盖:for + 范围、Vec 积累)
- 任务 4:数组统计用 for 遍历 + if 分支,并用 if 表达式生成结论字符串(覆盖:for 迭代器遍历、if 表达式)
- 任务 5:用 while let 消费一个栈直到空,并统计弹出值的和(覆盖:while let 语法糖)
- 任务 6:二维查找用嵌套 for + 带标签 break 'outer(覆盖:标签 break 跳出多层)
- 任务 7:输入解析用 if let/let-else 风格处理失败(覆盖:if let、let-else)

实验环境:cargo new menu_tool,代码写入 src/main.rs;用 cargo run 交互测试,再准备一批非数字输入验证健壮性。

第一步 **一步:**写 read_u32 辅助函数(读一行、trim、parse),失败返回 Option;主菜单里用 let-else 风格处理

第二步 **二步:**先实现纯函数 is_prime 与 sieve,不碰任何 I/O——纯函数最好测,也最容易验证正确性

第三步 **三步:**实现任务 4 的统计,输出时用 {:>4} 对齐素数表格

第四步 **四步:**任务 6 的关键:给外层 for 加 'outer: 标签,内层 break 'outer 才生效

第五步 **五步:**把每个工具放在 match 的独立分支里,分支过长时提取成函数(生产习惯)

参考答案要点

```
// ===== 实验 3:菜单驱动的控制台工具集 =====
// 覆盖知识点:loop 无限循环与菜单(§3.5)、match 穷尽与 _ 通配(§3.3)、
//          if 表达式(§3.2)、if let / while let / let-else(§3.4)、
//          while 与提前返回(§3.7)、for + 范围遍历(§3.6)、
//          break 带值 / continue / 带标签 break 'outer'(§3.5)

use std::io::{self, Write};

/// 素数判断:while + 提前返回(C 的 goto 检查模式的现代替代)
fn is_prime(n: u32) -> bool {
    if n < 2 {
        return false;
    }
    if n == 2 {
        return true;
    }
    if n % 2 == 0 {
        return false;           // 偶数直接排除,提前返回
    }
    let mut i = 3;
    while i * i <= n {
        if n % i == 0 {
            return false;       // 提前返回:不需要跳转清理
        }
        i += 2;
    }
    true
}

/// 收集 [2, limit] 内所有素数:for + 范围遍历
fn sieve(limit: u32) -> Vec<u32> {
    let mut primes = Vec::new();
    for n in 2..=limit {
        if is_prime(n) {
            primes.push(n);
        }
    }
    primes
}

/// 从 stdin 读一行并解析为 u32,失败返回 None
fn read_u32() -> Option<u32> {
    let mut line = String::new();
    io::stdin().read_line(&mut line).ok()?; // ? 在 Option 上同样可用
    line.trim().parse().ok()
}

fn main() {
    // 主菜单:loop + match 驱动(无限循环的标准写法)
    loop {
        println!();
        println!("===== 工具菜单 =====");
        println!("(1) 素数筛选(2..=N)");
        println!("(2) 数组统计");
        println!("(3) 二维查找(标签 break)");
    }
}
```



```

println!("4) 退出");
print!("请选择:");
let _ = io::stdout().flush();

// let-else:输入无效则回菜单(覆盖:let-else 提前退出)
let Some(choice) = read_u32() else {
    println!("输入无效,请重新输入");
    continue;           // 回到菜单
};

match choice {
    1 => {
        println!("请输入上限 N:");
        let _ = io::stdout().flush();
        if let Some(n) = read_u32() { // if let:只有成功才继续
            let primes = sieve(n);
            println!("2 到 {n} 之间有 {} 个素数:", primes.len());
            for (idx, p) in primes.iter().enumerate() {
                print!("{p:>4}");
                if (idx + 1) % 10 == 0 {
                    println(); // 每行 10 个
                }
            }
            println();
        } else {
            println!("请输入整数");
        }
    }
    2 => {
        let data = [12, 5, 8, 99, 3, 42, 7];
        let mut max = data[0];
        let mut sum: i32 = 0;
        let mut evens = 0;
        for &x in &data { // 只读迭代器遍历
            if x > max { max = x; } // if 语句形式
            sum += x;
            if x % 2 == 0 { evens += 1; }
        }
        // while let:弹出直到栈空
        let mut stack = vec![1, 2, 3, 4];
        let mut popped_sum = 0;
        while let Some(v) = stack.pop() {
            popped_sum += v;
        }
        // if 表达式:生成结论字符串
        let verdict = if max > 50 { "最大值超过 50" } else { "最大值未超过 50" };
        println!("最大值 {max}, 总和 {sum}, 偶数个数 {evens}, 弹出和 {popped_sum}");
        println!("{verdict}");
    }
    3 => {
        // 带标签 break:一次性跳出两层循环(goto 的安全替代)
        let target = 7;
        let mut found: Option<(u32, u32)> = None;
        'outer: for row in 0..4 {
            for col in 0..4 {

```

```

        let v = row * 4 + col;
        if v == target {
            found = Some((row, col));
            break 'outer;      // 跳出外层 for
        }
    }
}

match found {
    Some((r, c)) => println!("找到 {target}:第 {r} 行第 {c} 列"),
    None => println!("未找到 {target}"),
}

}

4 => {
    println!("再见!");
    break;                      // 退出主菜单循环
}

_ => {
    println!("无效选项,请重试");
    continue;                  // 明确回到菜单
}

}

}

// break 带值演示:loop 直接算出一个结果
let mut n = 1;
let first = loop {
    n += 1;
    if n * n > 1000 {
        break n;
    }
};
println!("第一个平方大于 1000 的数是 {first}");
}

```

设计说明:菜单循环刻意把每种流程控制都放进真实场景——match 管分发(穷尽性保证任何选项都有去处)、if let 管输入解析(失败自然回退)、while let 管栈消费、'outer' 管多层跳出、break 带值管"循环求值"。把任务 1 的分支改成一个没有 _ 的 match 试试:编译器立刻报非穷尽——这就是本章学的核心价值。

下一章预告:第4章 函数与方法——C 的函数指针时代如何被 Rust 的函数、方法、闭包全面取代;参数传递的"所有权移动"与"借用"二分法;Fn/FnMut/FnOnce 三态闭包;以及 Result 错误返回约定。准备好了吗?第5章的钥匙就藏在参数传递里。